

T20 5 QUESTIONS

```
import pandas as pd
import numpy as np
```

```
from google.colab import files
uploaded = files.upload()
```

```
df = pd.read_csv("International_T20_Data.csv")
df.head()
#1)Rename all the column names to appropriate
names.(Example: meta.created → created_date)
df.columns = df.columns.str.replace('.', '_')
df.columns
```

```
#2)Find the top three venues which hosted the greatest
number of matches.
top_venues = df['info_venue'].value_counts().head(3)
print("Top 3 Venues:")
print(top_venues)
```

```
#3)Find the pair of cricket teams who played the most
number of matches against each other
import ast
```

```
df['info_teams'] =
df['info_teams'].apply(ast.literal_eval)
df['team1'] = df['info_teams'].apply(lambda x: x[0])
df['team2'] = df['info_teams'].apply(lambda x: x[1])
df['team_pair'] = df.apply(lambda x:
tuple(sorted([x['team1'], x['team2']])), axis=1)
```

```
most_matches_pair =
df['team_pair'].value_counts().head(1)
```

```
print("Teams that played the most matches against each
other:")
```

```
print(most_matches_pair)
```

#4) Print the top five teams by their win percentages

```
matches_played = pd.concat([df['team1'],  
df['team2']]).value_counts()
```

```
matches_won = df['info_outcome_winner'].value_counts()
```

```
win_percentage = (matches_won / matches_played) * 100
```

```
top5_teams =
```

```
win_percentage.sort_values(ascending=False).head(5)
```

```
print("Top 5 Teams by Win Percentage:")
```

```
print(top5_teams)
```

#5) Function to get the scorecard of each match

```
def get_scorecard(match_index):
```

```
    import pandas as pd
```

```
    match = df.iloc[match_index]
```

```
    innings_data = match['innings']
```

```
    scorecards = []
```

```
    for inn in innings_data:
```

```
        innings = list(inn.values())[0]
```

```
        team = innings['team']
```

```
        deliveries = innings['deliveries']
```

```
        balls = []
```

```
        # flatten deliveries
```

```
        for d in deliveries:
```

```
            ball = list(d.values())[0]
```

```
            runs = ball['runs']
```

```
            balls.append({
```

```

        'batsman': ball['batsman'],
        'bowler': ball['bowler'],
        'runs_batsman': runs['batsman'],
        'runs_total': runs['total']
    })

df_innings = pd.DataFrame(balls)

# top batsmen
top_batsmen = (
    df_innings.groupby('batsman')['runs_batsman']
    .sum()
    .sort_values(ascending=False)
    .head(4)
)

# top bowlers
top_bowlers = (
    df_innings.groupby('bowler')['runs_total']
    .sum()
    .sort_values()
    .head(4)
)

scorecard = pd.concat([top_batsmen, top_bowlers],
axis=1)

print("\nTeam:", team)
print(scorecard)

scorecards.append(scorecard)

return scorecards

get_scorecard(0)

```

Applying diff libraries-

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from google.colab import files
uploaded = files.upload()
```

Saving International_T20_Data.csv to International_T20_Data.csv

pandas-

```
df = pd.read_csv("International_T20_Data.csv")
```

```
# 1. head()
# Use: Shows first rows of dataset
# Syntax: df.head(n)
# =====
print("First 5 rows:")
print(df.head())
```

First 5 rows:

		innings	meta.data_version	\
0	[{'1st innings': {'team': 'Australia', 'delive...		0.9	
1	[{'1st innings': {'team': 'Australia', 'delive...		0.9	
2	[{'1st innings': {'team': 'Australia', 'delive...		0.9	
3	[{'1st innings': {'team': 'Hong Kong', 'delive...		0.9	
4	[{'1st innings': {'team': 'Zimbabwe', 'deliver...		0.9	

	meta.created	meta.revision	info.dates	info.gender	\
0	2017-02-18	2	[datetime.date(2017, 2, 17)]	male	
1	2017-02-19	2	[datetime.date(2017, 2, 19)]	male	
2	2017-02-23	1	[datetime.date(2017, 2, 22)]	male	
3	2016-09-12	1	[datetime.date(2016, 9, 5)]	male	
4	2016-06-19	1	[datetime.date(2016, 6, 18)]	male	

	info.match_type	info.outcome.by.wickets	info.outcome.winner	info.overs	\
0	T20	5.0	Sri Lanka	20	
1	T20	2.0	Sri Lanka	20	
2	T20	NaN	Australia	20	
3	T20	NaN	Hong Kong	20	
4	T20	NaN	Zimbabwe	20	

	... info.outcome.by.runs	info.match_type_number	info.neutral_venue	\
0	...	NaN	NaN	NaN
1	...	NaN	NaN	NaN

2	...	41.0	NaN	NaN
3	...	40.0	NaN	NaN
4	...	2.0	NaN	NaN

	info.outcome.method	info.outcome.result	info.outcome.eliminator	\
0	NaN	NaN	NaN	
1	NaN	NaN	NaN	
2	NaN	NaN	NaN	
3	NaN	NaN	NaN	
4	NaN	NaN	NaN	

	info.supersubs.New Zealand	info.supersubs.South Africa	info.bowl_out	\
0	NaN	NaN	NaN	
1	NaN	NaN	NaN	
2	NaN	NaN	NaN	
3	NaN	NaN	NaN	
4	NaN	NaN	NaN	

	info.outcome.bowl_out
0	NaN
1	NaN
2	NaN
3	NaN
4	NaN

[5 rows x 27 columns]

```
# 2. tail()
# Use: Shows last rows of dataset
# Syntax: df.tail(n)
# =====
print("Last 5 rows:")
print(df.tail())
```

Last 5 rows:

		innings	meta.data_version
1412	[{'1st innings': {'team': 'Sri Lanka', 'delive...		0.9
1413	[{'1st innings': {'team': 'Bangladesh', 'deliv...		0.9
1414	[{'1st innings': {'team': 'Netherlands', 'deli...		0.9
1415	[{'1st innings': {'team': 'Australia', 'delive...		0.9
1416	[{'1st innings': {'team': 'Sri Lanka', 'delive...		0.9

	meta.created	meta.revision	info.dates	info.gender
1412	2016-03-05	2	[datetime.date(2016, 3, 4)]	male
1413	2016-03-08	1	[datetime.date(2016, 3, 6)]	male
1414	2016-02-03	1	[datetime.date(2016, 2, 3)]	male
1415	2016-09-12	1	[datetime.date(2016, 9, 6)]	male
1416	2016-09-12	1	[datetime.date(2016, 9, 9)]	male

```

        info.match_type  info.outcome.by.wickets  info.outcome.winner
info.overs  \
1412          T20          6.0          Pakistan
20
1413          T20          8.0          India
20
1414          T20          NaN          Netherlands
20
1415          T20          NaN          Australia
20
1416          T20          4.0          Australia
20

        ...  info.outcome.by.runs  info.match_type_number  info.neutral_venue  \
1412  ...          NaN          NaN          1.0
1413  ...          NaN          NaN          NaN
1414  ...          84.0          NaN          NaN
1415  ...          85.0          NaN          NaN
1416  ...          NaN          NaN          NaN

        info.outcome.method  info.outcome.result  info.outcome.eliminator  \
1412          NaN          NaN          NaN
1413          NaN          NaN          NaN
1414          NaN          NaN          NaN
1415          NaN          NaN          NaN
1416          NaN          NaN          NaN

        info.supersubs.New Zealand  info.supersubs.South Africa  info.bowl_out
\
1412          NaN          NaN          NaN
1413          NaN          NaN          NaN
1414          NaN          NaN          NaN
1415          NaN          NaN          NaN
1416          NaN          NaN          NaN

        info.outcome.bowl_out
1412          NaN
1413          NaN
1414          NaN
1415          NaN
1416          NaN

```

[5 rows x 27 columns]

```

# 3. info()
# Use: Gives column names, datatype, null values
# Syntax: df.info()
# =====
print("Dataset Info:")
df.info()

```

Dataset Info:

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1417 entries, 0 to 1416
Data columns (total 27 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   innings                              1417 non-null   object
 1   meta.data_version                    1417 non-null   float64
 2   meta.created                         1417 non-null   object
 3   meta.revision                        1417 non-null   int64
 4   info.dates                           1417 non-null   object
 5   info.gender                          1417 non-null   object
 6   info.match_type                      1417 non-null   object
 7   info.outcome.by.wickets              651 non-null    float64
 8   info.outcome.winner                  1372 non-null   object
 9   info.overs                           1417 non-null   int64
10   info.player_of_match                 1255 non-null   object
11   info.teams                           1417 non-null   object
12   info.toss.decision                   1417 non-null   object
13   info.toss.winner                     1417 non-null   object
14   info.umpires                         1395 non-null   object
15   info.venue                           1417 non-null   object
16   info.city                            1228 non-null   object
17   info.outcome.by.runs                 721 non-null    float64
18   info.match_type_number               684 non-null    float64
19   info.neutral_venue                  328 non-null    float64
20   info.outcome.method                  51 non-null     object
21   info.outcome.result                  45 non-null     object
22   info.outcome.eliminator              16 non-null     object
23   info.supersubs.New Zealand           1 non-null      object
24   info.supersubs.South Africa          1 non-null      object
25   info.bowl_out                        2 non-null      object
26   info.outcome.bowl_out                2 non-null      object
dtypes: float64(5), int64(2), object(20)
memory usage: 299.0+ KB

```

```

# 4. describe()
# Use: Statistical summary of numeric columns
# Syntax: df.describe()
# =====
print("Statistical Summary:")
print(df.describe())

Statistical Summary:
      meta.data_version  meta.revision  info.outcome.by.wickets
info.overs \
count      1.417000e+03      1417.000000      651.000000
1417.000000
mean        9.000000e-01        1.370501        6.476190
20.063514
std         5.108829e-15        0.575198        1.987802
1.379399

```

min	9.000000e-01	1.000000	1.000000
20.000000			
25%	9.000000e-01	1.000000	5.000000
20.000000			
50%	9.000000e-01	1.000000	7.000000
20.000000			
75%	9.000000e-01	2.000000	8.000000
20.000000			
max	9.000000e-01	5.000000	10.000000
50.000000			

	info.outcome.by.runs	info.match_type_number	info.neutral_venue
count	721.000000	684.000000	328.0
mean	39.088766	748.774854	1.0
std	33.385757	190.050264	0.0
min	1.000000	380.000000	1.0
25%	13.000000	627.000000	1.0
50%	30.000000	746.500000	1.0
75%	56.000000	883.250000	1.0
max	249.000000	1104.000000	1.0

```
# 5. isnull()
# Use: Detects missing values
# Syntax: df.isnull()
# =====
print("Missing Values Count:")
print(df.isnull().sum())
```

```
Missing Values Count:
innings                                0
meta.data_version                      0
meta.created                           0
meta.revision                           0
info.dates                             0
info.gender                             0
info.match_type                         0
info.outcome.by.wickets                 766
info.outcome.winner                     45
info.overs                              0
info.player_of_match                   162
info.teams                              0
info.toss.decision                      0
info.toss.winner                        0
info.umpires                            22
info.venue                              0
info.city                               189
info.outcome.by.runs                    696
info.match_type_number                  733
info.neutral_venue                      1089
info.outcome.method                     1366
info.outcome.result                     1372
info.outcome.eliminator                 1401
```



```
info.supersubs.New Zealand      1416
info.supersubs.South Africa     1416
info.bowl_out                   1415
info.outcome.bowl_out           1415
dtype: int64
```

```
# 6. dropna()
# Use: Removes missing values
# Syntax: df.dropna()
# =====
clean_df = df[['info.outcome.by.runs', 'info.outcome.by.wickets']].dropna()
print("After Dropna:")
print(clean_df.head())
```

After Dropna:
Empty DataFrame
Columns: [info.outcome.by.runs, info.outcome.by.wickets]
Index: []

```
# 7. fillna()
# Use: Replaces missing values
# Syntax: df.fillna(value)
# =====
filled_runs = df['info.outcome.by.runs'].fillna(0)
print("Filled Missing Runs:")
print(filled_runs.head())
```

Filled Missing Runs:

0	0.0
1	0.0
2	41.0
3	40.0
4	2.0

Name: info.outcome.by.runs, dtype: float64

```
# 8. sort_values()
# Use: Sorts data by column
# Syntax: df.sort_values('column')
# =====
sorted_runs = df.sort_values('info.outcome.by.runs')
print("Sorted Runs:")
print(sorted_runs[['info.outcome.by.runs']].head())
```

Sorted Runs:

	info.outcome.by.runs
138	1.0
843	1.0
910	1.0
185	1.0

1083

1.0

```
# 9. value_counts()
# Use: Counts unique values in categorical column
# Syntax: df['column'].value_counts()
# =====
print("Gender Count:")
print(df['info.gender'].value_counts())
```

```
Gender Count:
info.gender
male      951
female    466
Name: count, dtype: int64
```

```
# 10. groupby()
# Use: Groups data and performs aggregation
# Syntax: df.groupby('column').mean()
# =====
grouped = df.groupby('info.gender')['info.outcome.by.runs'].mean()
print("Average Runs by Gender:")
print(grouped)
```

```
Average Runs by Gender:
info.gender
female    44.163180
male      36.572614
Name: info.outcome.by.runs, dtype: float64
```

```
# 11. loc[]
# Use: Select rows/columns by label
# Syntax: df.loc[row, column]
# =====
print("Using loc:")
print(df.loc[0:5, ['info.gender', 'info.outcome.by.runs']])
```

```
Using loc:
  info.gender  info.outcome.by.runs
0         male                  NaN
1         male                  NaN
2         male                 41.0
3         male                 40.0
4         male                  2.0
5         male                  NaN
```

```
# 12. iloc[]
```

```
# Use: Select rows/columns by index position
# Syntax: df.iloc[row, column]
# =====
print("Using iloc:")
print(df.iloc[0:5, 0:3])
```

Using iloc:

	innings	meta.data_version	\
0	['1st innings': {'team': 'Australia', 'delive...	0.9	
1	['1st innings': {'team': 'Australia', 'delive...	0.9	
2	['1st innings': {'team': 'Australia', 'delive...	0.9	
3	['1st innings': {'team': 'Hong Kong', 'delive...	0.9	
4	['1st innings': {'team': 'Zimbabwe', 'deliver...	0.9	

	meta.created
0	2017-02-18
1	2017-02-19
2	2017-02-23
3	2016-09-12
4	2016-06-19

```
# 13. rename()
# Use: Renames columns
# Syntax: df.rename(columns={'old':'new'})
# =====
renamed_df = df.rename(columns={'info.gender':'Gender'})
print("Renamed Column:")
print(renamed_df[['Gender']].head())
```

Renamed Column:

	Gender
0	male
1	male
2	male
3	male
4	male

```
# 14. drop()
# Use: Removes column or row
# Syntax: df.drop('column', axis=1)
# =====
dropped_df = df.drop('info.gender', axis=1)
print("After Dropping Column:")
print(dropped_df.head())
```

After Dropping Column:

	innings	meta.data_version	\
0	['1st innings': {'team': 'Australia', 'delive...	0.9	
1	['1st innings': {'team': 'Australia', 'delive...	0.9	

```

2  [{'1st innings': {'team': 'Australia', 'delive...           0.9
3  [{'1st innings': {'team': 'Hong Kong', 'delive...         0.9
4  [{'1st innings': {'team': 'Zimbabwe', 'deliver...         0.9

```

```

meta.created meta.revision info.dates info.match_type
\
0  2017-02-18          2  [datetime.date(2017, 2, 17)]      T20
1  2017-02-19          2  [datetime.date(2017, 2, 19)]      T20
2  2017-02-23          1  [datetime.date(2017, 2, 22)]      T20
3  2016-09-12          1  [datetime.date(2016, 9, 5)]       T20
4  2016-06-19          1  [datetime.date(2016, 6, 18)]      T20

```

```

info.outcome.by.wickets info.outcome.winner info.overs \
0          5.0          Sri Lanka          20
1          2.0          Sri Lanka          20
2          NaN          Australia          20
3          NaN          Hong Kong          20
4          NaN          Zimbabwe          20

```

```

info.player_of_match ... info.outcome.by.runs info.match_type_number \
0  ['DAS Gunaratne'] ...          NaN          NaN
1  ['DAS Gunaratne'] ...          NaN          NaN
2          ['A Zampa'] ...          41.0          NaN
3          NaN ...          40.0          NaN
4  ['E Chigumbura'] ...          2.0          NaN

```

```

info.neutral_venue info.outcome.method info.outcome.result \
0          NaN          NaN          NaN
1          NaN          NaN          NaN
2          NaN          NaN          NaN
3          NaN          NaN          NaN
4          NaN          NaN          NaN

```

```

info.outcome.eliminator info.supersubs.New Zealand \
0          NaN          NaN
1          NaN          NaN
2          NaN          NaN
3          NaN          NaN
4          NaN          NaN

```

```

info.supersubs.South Africa info.bowl_out info.outcome.bowl_out
0          NaN          NaN          NaN
1          NaN          NaN          NaN
2          NaN          NaN          NaN
3          NaN          NaN          NaN
4          NaN          NaN          NaN

```

```
[5 rows x 26 columns]
```

```

# 15. corr()
# Use: Finds correlation between numeric columns
# Syntax: df.corr()

```

```
# =====
corr_data =
df[['info.outcome.by.runs', 'info.outcome.by.wickets', 'info.overs']].corr()
print("Correlation Matrix:")
print(corr_data)
```

Correlation Matrix:

	info.outcome.by.runs	info.outcome.by.wickets	\
info.outcome.by.runs	1.000000		NaN
info.outcome.by.wickets		1.000000	NaN
info.overs	0.021201	0.010344	

	info.overs
info.outcome.by.runs	0.021201
info.outcome.by.wickets	0.010344
info.overs	1.000000

Matplotlib

```
# 1. plt.plot()
# Use: Draws line graph to show trend between two variables
# Syntax: plt.plot(x, y)
# =====
temp =
df[['info.match_type_number', 'info.outcome.by.runs']].dropna()
.sort_values('info.match_type_number')
```

```
plt.figure(figsize=(6,4))
plt.plot(temp['info.match_type_number'],
temp['info.outcome.by.runs'])
plt.title("Line Plot: Match Number vs Runs")
plt.xlabel("Match Number")
plt.ylabel("Runs")
plt.show()
```

```
# 2. plt.scatter()
# Use: Shows relationship between two numerical columns
# Syntax: plt.scatter(x, y)
# =====
temp =
df[['info.outcome.by.runs', 'info.outcome.by.wickets']].dropna(
)
```

```

plt.figure(figsize=(6,4))
plt.scatter(temp['info.outcome.by.runs'],
temp['info.outcome.by.wickets'])
plt.title("Scatter Plot: Runs vs Wickets")
plt.xlabel("Runs")
plt.ylabel("Wickets")
plt.show()
# 3. plt.bar()
# Use: Compares categorical values
# Syntax: plt.bar(x, height)
# =====
temp = df['info.gender'].value_counts()

plt.figure(figsize=(6,4))
plt.bar(temp.index, temp.values)
plt.title("Bar Plot: Gender Count")
plt.xlabel("Gender")
plt.ylabel("Count")
plt.show()
# 4. plt.hist()
# Use: Shows frequency distribution
# Syntax: plt.hist(data)
# =====
plt.figure(figsize=(6,4))
plt.hist(df['info.outcome.by.runs'].dropna(), bins=20)
plt.title("Histogram of Runs")
plt.xlabel("Runs")
plt.ylabel("Frequency")
plt.show()

# 5. plt.boxplot()
# Use: Detects spread and outliers
# Syntax: plt.boxplot(data)
# =====
plt.figure(figsize=(6,4))
plt.boxplot(df['info.outcome.by.wickets'].dropna())
plt.title("Boxplot of Wickets")
plt.show()

```

```

# 6. plt.pie()
# Use: Shows category proportion
# Syntax: plt.pie(values, labels=labels)
# =====
temp = df['info.toss.decision'].value_counts()

plt.figure(figsize=(6,4))
plt.pie(temp.values, labels=temp.index, autopct='%1.1f%%')
plt.title("Pie Chart of Toss Decision")
plt.show()

# 7. plt.stem()
# Use: Displays discrete data points
# Syntax: plt.stem(x, y)
# =====
temp =
df[['info.match_type_number', 'info.outcome.by.runs']].dropna()
.head(20)

plt.figure(figsize=(6,4))
plt.stem(temp['info.match_type_number'],
temp['info.outcome.by.runs'])
plt.title("Stem Plot")
plt.show()

# 8. plt.step()
# Use: Shows stepwise trend
# Syntax: plt.step(x, y)
# =====
plt.figure(figsize=(6,4))
plt.step(temp['info.match_type_number'],
temp['info.outcome.by.runs'])
plt.title("Step Plot")
plt.show()

# 9. plt.fill_between()
# Use: Fills area under line
# Syntax: plt.fill_between(x, y)
# =====
plt.figure(figsize=(6,4))
plt.fill_between(temp['info.match_type_number'],
temp['info.outcome.by.runs'])

```

```

plt.title("Fill Between Plot")
plt.show()
# 10. plt.errorbar()
# Use: Shows values with error bars
# Syntax: plt.errorbar(x, y, yerr=error)
# =====
plt.figure(figsize=(6,4))
plt.errorbar(temp['info.match_type_number'],
temp['info.outcome.by.runs'], yerr=2)
plt.title("Error Bar Plot")
plt.show()
# 11. plt.stackplot()
# Use: Shows cumulative stacked data
# Syntax: plt.stackplot(x, y1, y2)
# =====
temp =
df[['info.match_type_number','info.outcome.by.runs','info.outc
ome.by.wickets']].dropna().head(20)

plt.figure(figsize=(6,4))
plt.stackplot(temp['info.match_type_number'],
temp['info.outcome.by.runs'],
temp['info.outcome.by.wickets'])
plt.title("Stack Plot")
plt.show()
# 12. plt.vlines()
# Use: Draws vertical lines
# Syntax: plt.vlines(x, ymin, ymax)
# =====
plt.figure(figsize=(6,4))
plt.vlines(temp['info.match_type_number'], 0,
temp['info.outcome.by.runs'])
plt.title("Vertical Lines Plot")
plt.show()
# 13. plt.hlines()
# Use: Draws horizontal lines
# Syntax: plt.hlines(y, xmin, xmax)
# =====
plt.figure(figsize=(6,4))

```



```

plt.hlines(temp['info.outcome.by.runs'], 0,
temp['info.match_type_number'])
plt.title("Horizontal Lines Plot")
plt.show()
# 14. plt.hexbin()
# Use: Density plot for large scatter data
# Syntax: plt.hexbin(x, y)
# =====
temp =
df[['info.outcome.by.runs','info.outcome.by.wickets']].dropna(
)

plt.figure(figsize=(6,4))
plt.hexbin(temp['info.outcome.by.runs'],
temp['info.outcome.by.wickets'])
plt.title("Hexbin Plot")
plt.show()
# 15. plt.eventplot()
# Use: Shows occurrence of events
# Syntax: plt.eventplot(data)
# =====
plt.figure(figsize=(6,4))
plt.eventplot(df['info.outcome.by.runs'].dropna().head(30))
plt.title("Event Plot")
plt.show()

```

NUMPY-

```

# 1. np.array()
# Use: Converts data into NumPy array
# Syntax: np.array(data)
# =====
runs_array = np.array(runs)
print("Array:", runs_array[:10])

# 2. np.mean()
# Use: Calculates average value
# Syntax: np.mean(array)
# =====
print("Mean Runs:", np.mean(runs))

```

```

# 3. np.median()
# Use: Finds middle value
# Syntax: np.median(array)
# =====
print("Median Runs:", np.median(runs))

# 4. np.std()
# Use: Calculates standard deviation
# Syntax: np.std(array)
# =====
print("Standard Deviation:", np.std(runs))

# 5. np.var()
# Use: Calculates variance
# Syntax: np.var(array)
# =====
print("Variance:", np.var(runs))

# 6. np.min()
# Use: Finds minimum value
# Syntax: np.min(array)
# =====
print("Minimum Runs:", np.min(runs))

# 7. np.max()
# Use: Finds maximum value
# Syntax: np.max(array)
# =====
print("Maximum Runs:", np.max(runs))

# 8. np.sum()
# Use: Calculates total sum
# Syntax: np.sum(array)
# =====
print("Total Runs:", np.sum(runs))

```

```

# 9. np.sort()
# Use: Sorts data in ascending order
# Syntax: np.sort(array)
# =====
print("Sorted Runs:", np.sort(runs)[:10])

# 10. np.unique()
# Use: Finds unique values
# Syntax: np.unique(array)
# =====
print("Unique Wickets:", np.unique(wickets))

# 11. np.sqrt()
# Use: Calculates square root
# Syntax: np.sqrt(array)
# =====
print("Square Root of Runs:", np.sqrt(runs[:10]))

# 12. np.log()
# Use: Calculates natural logarithm
# Syntax: np.log(array)
# =====
positive_runs = runs[runs > 0]
print("Log of Runs:", np.log(positive_runs[:10]))

# 13. np.corrcoef()
# Use: Finds correlation between two arrays
# Syntax: np.corrcoef(array1, array2)
# =====
min_len = min(len(runs), len(wickets))
print("Correlation:\n", np.corrcoef(runs[:min_len],
wickets[:min_len]))

# 14. np.percentile()
# Use: Finds percentile value
# Syntax: np.percentile(array, percentile)
# =====
print("75th Percentile Runs:", np.percentile(runs, 75))

# 15. np.where()
# Use: Conditional filtering

```

```
# Syntax: np.where(condition)
# =====
print("Runs > 50:", np.where(runs > 50))
```

-seaborn

```
# 1. histplot()
# Use: Shows frequency distribution of a numerical column
# Syntax: sns.histplot(data=df, x='column')
# =====
plt.figure(figsize=(6,4))
sns.histplot(data=df, x='info.outcome.by.runs', bins=20)
plt.title("Distribution of Runs")
plt.show()

# 2. boxplot()
# Use: Detects spread and outliers in numerical data
# Syntax: sns.boxplot(data=df, x='column')
# =====
plt.figure(figsize=(6,4))
sns.boxplot(data=df, x='info.outcome.by.wickets')
plt.title("Boxplot of Wickets")
plt.show()

# 3. scatterplot()
# Use: Shows relationship between two numerical variables
# Syntax: sns.scatterplot(data=df, x='col1', y='col2')
# =====
plt.figure(figsize=(6,4))
sns.scatterplot(data=df, x='info.outcome.by.runs',
y='info.outcome.by.wickets')
plt.title("Runs vs Wickets")
plt.show()

# 4. lineplot()
# Use: Displays trend over ordered values
# Syntax: sns.lineplot(data=df, x='col1', y='col2')
# =====
temp =
df[['info.match_type_number', 'info.outcome.by.runs']].dropna()
.sort_values('info.match_type_number')
plt.figure(figsize=(6,4))
```

```

sns.lineplot(data=temp, x='info.match_type_number',
y='info.outcome.by.runs')
plt.title("Match Number vs Runs")
plt.show()

# 5. barplot()
# Use: Compares average values across categories
# Syntax: sns.barplot(data=df, x='category', y='value')
# =====
temp = df[['info.gender', 'info.outcome.by.runs']].dropna()
plt.figure(figsize=(6,4))
sns.barplot(data=temp, x='info.gender',
y='info.outcome.by.runs')
plt.title("Average Runs by Gender")
plt.show()

# 6. countplot()
# Use: Counts occurrences of categories
# Syntax: sns.countplot(data=df, x='category')
# =====
plt.figure(figsize=(6,4))
sns.countplot(data=df, x='info.toss.decision')
plt.title("Toss Decision Count")
plt.show()

# 7. violinplot()
# Use: Shows distribution + density of data
# Syntax: sns.violinplot(data=df, x='category', y='value')
# =====
temp = df[['info.gender', 'info.outcome.by.runs']].dropna()
plt.figure(figsize=(6,4))
sns.violinplot(data=temp, x='info.gender',
y='info.outcome.by.runs')
plt.title("Runs Distribution by Gender")
plt.show()

# 8. stripplot()
# Use: Displays all individual observations
# Syntax: sns.stripplot(data=df, x='category', y='value')
# =====
plt.figure(figsize=(6,4))

```

```

sns.stripplot(data=temp, x='info.gender',
y='info.outcome.by.runs')
plt.title("Stripplot of Runs by Gender")
plt.show()
# 9. swarmplot()
# Use: Prevents overlapping points in categorical scatter
# Syntax: sns.swarmplot(data=df, x='category', y='value')
# =====
plt.figure(figsize=(6,4))
sns.swarmplot(data=temp.sample(200), x='info.gender',
y='info.outcome.by.runs')
plt.title("Swarmplot of Runs")
plt.show()
# 10. heatmap()
# Use: Shows correlation matrix visually
# Syntax: sns.heatmap(df.corr())
# =====
plt.figure(figsize=(6,4))
corr =
df[['info.outcome.by.runs','info.outcome.by.wickets','info.overs',
'info.match_type_number']].corr()
sns.heatmap(corr, annot=True)
plt.title("Correlation Heatmap")
plt.show()

# 11. pairplot()
# Use: Shows pairwise relationships among numeric columns
# Syntax: sns.pairplot(df[['col1','col2']])
# =====
sns.pairplot(df[['info.outcome.by.runs','info.outcome.by.wickets',
'info.overs']].dropna())
plt.show()

# 12. kdeplot()
# Use: Shows smooth probability density curve
# Syntax: sns.kdeplot(data=df, x='column')
# =====
plt.figure(figsize=(6,4))
sns.kdeplot(data=df, x='info.outcome.by.runs')

```

```

plt.title("Density Plot of Runs")
plt.show()

# 13. regplot()
# Use: Shows regression trend between two variables
# Syntax: sns.regplot(data=df, x='col1', y='col2')
# =====
temp =
df[['info.outcome.by.runs', 'info.outcome.by.wickets']].dropna(
)
plt.figure(figsize=(6,4))
sns.regplot(data=temp, x='info.outcome.by.runs',
y='info.outcome.by.wickets')
plt.title("Regression Plot")
plt.show()

# 14. catplot()
# Use: General categorical plotting function
# Syntax: sns.catplot(data=df, x='category', y='value',
kind='box')
# =====

temp_cat = df[['info.gender',
'info.outcome.by.runs']].dropna()

sns.catplot(
    data=temp_cat,
    x='info.gender',
    y='info.outcome.by.runs',
    kind='box'
)

plt.title("Catplot of Runs by Gender")
plt.show()

# 15. displot()
# Use: Distribution plot with histogram/kde
# Syntax: sns.displot(data=df, x='column')
# =====
sns.displot(data=df, x='info.outcome.by.runs')

```

```
plt.show()
```